

Title: Deterministic Rank-Based Encoding for Entropy-Optimal Data-to-Location Mapping

Author: Trent M. Wyatt

Abstract

We introduce a novel, entropy-aligned model for data storage and retrieval that maps data directly to physical or virtual locations using reversible, bijective encoding. By integrating symbol frequency analysis with rank/unrank transformations and structured storage offsets, the system enables constant-time traversal, optimal compression density, and minimal mapping overhead. We formalize four primitive operations, demonstrate entropy efficiency compared to traditional schemes, and outline real-world applications in blockchain traversal, PKI indexing, and reversible data storage.

1. Introduction

Most modern storage architectures treat data as opaque blobs, requiring indexing, hashing, or lookup structures to manage retrieval. This leads to entropy loss, access overhead, and fragmentation. We present a method that directly maps data content to location via a deterministic, bijective transformation based on symbol set encoding and ranking. This scheme is domain-adaptive, invertible, and computationally efficient.

2. Primitives

We define four core primitives:

<code>analyze(data):</code>	Infer the optimal symbol encoding model from the input domain
<code>rank(data):</code>	Map data to an integer offset using rank in the encoding model
<code>unrank(offset):</code>	Reconstruct the original data from the offset
<code>store(offset):</code>	Map offset to physical or virtual storage address

These form a minimal reversible interface for structured, high-density data systems.

3. Theoretical Foundation

3.1 Rank/Unrank Over Symbol Alphabets

Given a character set Σ of size k and a fixed-length string $s \in \Sigma^n$, we can treat each s_i as a base- k digit:

$$\text{rank}(s) = \sum_{i=0}^{n-1} \text{val}(s_i) \cdot k^{n-i-1}$$

This results in a unique integer $r \in [0, k^n)$, and the process is invertible via unranking (modulo/divide).

3.2 Entropy Preservation

If H is the entropy per symbol (in bits), then total entropy is:

$$H_{total} = n \cdot H \leq \log_2(k^n) = n \cdot \log_2 k$$

By choosing symbol encodings that match the input distribution (via `analyze()`), we achieve entropy bounds equivalent to Huffman or ANS.

4. Storage Model

Offsets from `rank(data)` serve directly as addresses or keys:

- Memory-mapped files: offset = byte index
- Database keys: offset = deterministic primary key
- Blockchain traversal: offset = ordinal block ID

No secondary indexing is required.

5. Performance

Metric	Property
Entropy	Matched to input via adaptive model
Access time	O(1) rank/unrank
Storage	Fully deterministic, minimal mapping
Traversal	Arithmetic, not pointer-based
Collisions	None (bijective mapping)

6. Applications

- **PKI Systems:** Content-derived, collision-free public key indexing
 - **Dense Blockchains:** Rank-based traversal and verification
 - **Compressible File Systems:** Location as a function of content
 - **Zero-Knowledge Proof Systems:** Deterministic proof path indexing
-

7. Conclusion

We have presented a theoretically grounded, practically efficient method for data-to-location mapping that achieves optimal entropy utilization, reversible storage, and O(1) access speed. Through the combination of

adaptive modeling and rank-based transformation, the system balances compression, access, and determinism in a manner superior to traditional storage paradigms.

Future work will extend to multivariate symbol streams, cross-domain traversal, and native OS/kernel-level support.

Contact: robotics@trentwyatt.com